I review the empirical pathologies of deep reinforcement learning, their causes, and the diagnostic tools that expose them.

0.1 The Moving Target Problem

In supervised learning, the loss function is a *fixed function* of the training data and model parameters. Deep reinforcement learning does not enjoy this property. Each gradient step moves both the current value estimates and the targets simultaneously, creating a "nonstationary" optimization landscape. The target network heuristic introduced by **?** slows target drift by periodically freezing a copy of the network, but does not eliminate it. Therefore Bellman residual is a poor proxy for the accuracy of the value function.

? formalize this observation. Let Q^π denote the true value function for policy π, let $\Delta(s,a) = Q(s,a) - Q^\pi(s,a)$ denote the value error, and let $\varepsilon(s,a) = Q(s,a) - (r + \gamma\mathbb{E}_{s',a'}[Q(s',a')])$ denote the Bellman error. Substituting the definition of Q^π yields the key identity: $\varepsilon(s,a) = \Delta(s,a) - \gamma\mathbb{E}_{s',a'}[\Delta(s',a')]$. The Bellman error is a *difference* of value errors at consecutive states, not the value error itself. If the errors $\Delta(s,a)$ and $\Delta(s',a')$ are correlated across time—the network is wrong in the same direction at successive state-action pairs—they cancel in the difference and the Bellman error is small regardless of how large the individual errors are.[1]

The second failure mode is specific to finite datasets: **?**, Corollary 1 show that over an incomplete dataset, zero Bellman error is consistent with arbitrarily large value error, because the network can fit unobserved successor pairs to whatever values make the observed residuals vanish.[2]

The dual failure mode appears in policy gradient methods: **?** find that even when PPO's surrogate objective improves monotonically, episode return can plateau or decline, because the surrogate gradient is poorly aligned with the gradient of the true return.[3]

[1] In the extreme case, shifting all Q-values by the constant $c/(1-\gamma)$ leaves the Bellman error unchanged at zero while increasing value error by $c/(1-\gamma)$.

[2] The Bellman equation uniquely identifies Q^π when enforced over the entire MDP, but over an incomplete dataset it admits infinitely many solutions. Whenever a transition (s',a') that is reachable from the dataset is not itself in the dataset, the network is free to set $Q(s',a')$ at unobserved pairs to whatever value makes the residual vanish on the observed ones, unconstrained by any loss. A network can thus reach near-zero training loss while the value function remains arbitrarily inaccurate over the full state space.

[3] **?** examine the surrogate objective used in Proximal Policy Optimization (**?**). The PPO clipping mechanism is designed to keep policy updates within a trust region by bounding the probability ratio $\pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$. The gradient of the surrogate is poorly aligned with the gradient of the true return, particularly in later training phases where the policy has diverged from the behavior policy used to collect the replay data. The loss metric that practitioners monitor throughout training is measuring something other than what they care about.

0.2 The Reproducibility Crisis and Sensitivity to Random Seeds

? trained five leading policy gradient algorithms (PPO, TRPO, DDPG, TD3, SAC) on six MuJoCo benchmark environments, holding all hyperparameters fixed and varying only the random seed. The resulting learning curves from different seeds were non-overlapping: a seed that performed well under one algorithm performed comparably to a different algorithm's best seeds, making cross-algorithm comparison unreliable.[4] **?** quantify the damage: comparing point estimates from 5 runs per task on Atari 100k yields Type I error exceeding 50%, meaning a random noise injection appears beneficial in half of all comparisons.

? propose the *interquartile mean* (IQM) as a replacement for mean and median when comparing algorithms. The IQM discards the top and bottom 25% of runs before averaging, reducing sensitivity to outlier seeds. Using these tools and stratified bootstrap confidence intervals, **?** find that several widely-cited algorithmic improvements on Atari 100k vanish or reverse when statistical uncertainty is accounted for.[5][6]

0.3 Value Overestimation and Spikes

Q-learning uses the Bellman optimality update

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a'), \tag{1}$$

where the maximum is taken over the estimated Q-values of all actions at the successor state. **?** identify a positive bias intrinsic to this update: if the Q-value estimates contain noise with mean zero, the maximum over noisy estimates is biased upward by Jensen's inequality. An agent that uses a single network for both action selection ($\arg\max$) and value estimation ($\max Q$) systematically overestimates the values of every state it visits, biasing the Bellman target upward at every update step. The bias compounds through bootstrapping: overestimated targets produce overestimated updates, which produce fur-

[4] Differences as large as 2,000 points in final episode return arose from seed variation alone.

[5] They also introduce performance profiles, which plot the fraction of tasks and seeds where an algorithm achieves performance above a threshold τ, as τ varies from 0 to the maximum. Performance profiles reveal the full shape of the score distribution rather than collapsing it to a single statistic.

[6] The fragility extends to hyperparameters. **?** conduct a systematic study of hyperparameter sensitivity across 6 algorithms and 17 environments, finding that default hyperparameters from published papers perform competitively in the specific environments used in those papers but generalize poorly across environments. **?** synthesize these findings into an empirical design handbook, recommending at minimum 10 seeds per configuration, IQM-based comparisons, and preregistration of hyperparameter search protocols.

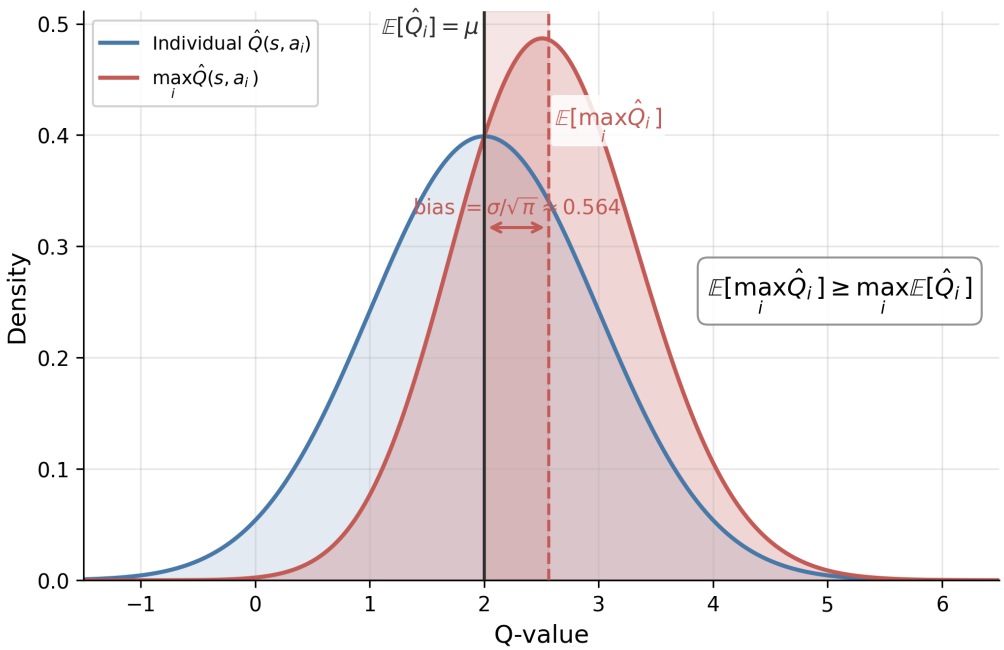

Figure 1: Overestimation bias from Jensen's inequality with $n = 2$ actions. Blue: density of individual $\hat{Q}(s, a_i) \sim \mathcal{N}(\mu, \sigma^2)$. Red: density of $\max_i \hat{Q}(s, a_i)$, shifted right by $\sigma/\sqrt{\pi} \approx 0.56$. The shaded gap is the bias. With $n = 100$ actions the bias exceeds 2.5σ.

ther overestimated targets.

? propose double Q-learning as a remedy: maintain two independent Q-networks Q_A and Q_B. Use Q_A to select the greedy action at s', but use Q_B to evaluate that action. Because the two networks are trained on different data, their errors are approximately independent, and the positive bias largely cancels. **?** implement this as Double DQN, using the online network for action selection and the periodically-frozen target network for evaluation. On 49 Atari games, Double DQN reduces overestimation by a factor of 3–5 and improves median performance by 20% relative to DQN.

? observe that Double DQN's correction is incomplete in continuous-action settings, where the target network and online network remain correlated through shared updates. They propose Clipped Double Q-learning: compute two Q-value estimates Q_1, Q_2 with separate networks trained on the same data, and use $y = r + \gamma \min(Q_1(s', a'), Q_2(s', a'))$ as the Bellman target. The minimum operator introduces pessimistic underestimation, which is conservative but avoids the explosive positive bias.[7]

DQN prevents outright divergence, but **?** find that *soft divergence*, defined as a

[7] **?** note that clipped double Q can cause excessive pessimism under high uncertainty, proposing optimistic actor-critic as a counterweight.

temporary spike in value estimates by more than 10% followed by recovery, occurs in the majority of DQN training runs across 57 Atari games.[8] The deadly triad does not announce itself as a training failure: the value estimates may spike and recover, leaving no visible trace in the loss curve while corrupting the policy.

? describe a continuous manifestation of the deadly triad: *implicit under-parameterization.*[9]

0.4 Plasticity Loss and Primacy Bias

A network that trains well at step $t = 100$ may be incapable of learning at step $t = 100{,}000$, even if the data quality at the later step is higher. **?** call this *capacity loss*: the network's ability to update its own weights degrades progressively during training, measured by the fraction of effective parameters and the network's ability to fit new random labels. **?** extend this to *plasticity loss*, identifying dead ReLU neurons, weight norm growth, and feature rank collapse as three distinct mechanisms, not all of which co-occur.

? identify *primacy bias* as a specific cause. Because the replay buffer is filled incrementally, early transitions are oversampled relative to later ones, and the network over-fits to early environment transitions, corrupting representations throughout the remainder of training.[10]

The proposed remedies fall into three categories: periodic resets, continual backpropagation, and architectural interventions.[11]

0.5 Implementation Dominates Algorithmic Innovation

? find that PPO with the clipping mechanism disabled performs indistinguishably from

[8] Larger networks diverge more frequently than smaller ones, counter to the usual intuition that more expressive models should generalize better.

[9] Neural networks trained with bootstrapped TD objectives progressively lose their effective rank, with fewer and fewer neurons contributing distinct directions in the representation. This rank collapse is silent (training loss continues to decrease) but the network's ability to represent new information degrades over time, a form of capacity loss distinct from but related to plasticity loss.

[10] **?** show that 100 initial "priming" steps of excessive gradient updates degrade a SAC agent's performance for hundreds of thousands of subsequent steps. **?** measure the fraction of dormant neurons (units with near-zero activation across the replay buffer) accumulating monotonically during training. **?** find that standard deep networks lose all plasticity within a few million gradient steps in continual learning tasks.

[11] Periodic resets reinitialize the last layers of the network while retaining the replay buffer, allowing the agent to forget overfit representations without discarding experience (**??**). Continual backpropagation replaces neurons with near-zero utility at each gradient step rather than waiting for a global reset (**?**). Architectural interventions—layer normalization (**?**), orthogonal initialization, spectral normalization—reduce the rate at which plasticity is lost by stabilizing gradient magnitudes and preventing weight norm growth.

the full algorithm; TRPO (**?**) with the same code-level additions matches PPO.[12] **?** identify observation normalization, orthogonal initialization, and learning rate annealing as the three choices accounting for most variance across 250,000 agents and 250 hyperparameter configurations. **?** catalog 37 implementation details required to reproduce PPO on Atari;[13] omitting any produces materially different results.

The most consequential implementation detail is the distinction between termination and truncation (**?**). In reinforcement learning, an episode can end for two distinct reasons: *termination*, where the environment reaches a natural absorbing state (the pole falls in CartPole, the robot falls in locomotion tasks), and *truncation*, where the episode is cut short by an external time limit. At a termination, the value of the successor state is zero: $V(s_{\text{term}}) = 0$. At a truncation, the episode is merely paused and the successor state has non-zero value: $V(s_{\text{trunc}}) \neq 0$. Treating truncated transitions as terminated substitutes zero for a non-zero bootstrap value at every time limit boundary, corrupting every Bellman update in the vicinity of episode boundaries. **?** show that this conflation degrades performance by 20–40% on standard MuJoCo benchmarks.[14]

0.6 Replay Buffer Pathologies and Reward Scaling

Experience replay (**?**) decouples the data collection and learning processes, allowing a single transition to be used for multiple gradient updates. The replay ratio (the number of gradient updates per environment step) governs the trade-off between sample efficiency and data staleness. **?** show that increasing the replay ratio beyond a modest threshold degrades performance.[15]

? propose prioritized experience replay (PER).[16] **?** revisit PER on large-scale Atari experiments and find that uniform sampling from a large enough buffer matches or outperforms PER, while being simpler to implement and tune.

[12] The non-clipping components that suffice are: observation normalization, reward normalization, value function clipping, global gradient norm clipping, orthogonal weight initialization, and the Adam optimizer.

[13] These include reward clipping to $[-1, 1]$, frame stacking to 4 frames, a specific episode termination convention, and a numerically stable normalization of the advantage estimate.

[14] The Gymnasium API (**?**) enforces the distinction by returning separate `terminated` and `truncated` flags, but most pre-2022 codebases conflate them in the `done` flag.

[15] At high replay ratios, the distribution shift between the current policy and the behavior policy that generated the stored data grows large enough to violate the off-policy assumptions of Q-learning. The relationship is non-monotone and environment-dependent, making replay buffer size a sensitive hyperparameter with no universal default.

[16] PER samples transitions with probability proportional to the magnitude of their TD error, on the argument that high-error transitions are the most informative.

Reward scaling introduces a separate class of failure modes. Standard DQN (**?**) clips rewards to $[-1, +1]$ across all environments to stabilize training. **?** observe that reward clipping changes the objective: clipped rewards make all positive events equivalent regardless of magnitude, so the agent learns to maximize the frequency of positive events rather than their cumulative value. This substitution can produce policies that are locally rational under the clipped reward but qualitatively suboptimal under the true reward. **?** propose PopArt as a remedy.[17]

? formalize reward hacking and show that any non-constant proxy reward can in principle be exploited by a sufficiently capable optimizer.[18]

0.7 Simulation Study: Bellman Error and Value Error in Offline Policy Evaluation

The MDP uses $s = (k, z)$ with k on a 50-point log-spaced capital grid and $z \in \{0.9, 1.1\}$ following a Markov chain with persistence 0.8. Actions are next-period capital choices on the same grid. Reward is $\log(zk^\alpha - k')$ with $\alpha = 0.36$, $\beta = 0.96$. Rewards are shifted by $-\bar{r}$ (mean reward over feasible pairs) to center Q^* near zero, which avoids initialization issues without altering the optimal policy. The offline dataset $\mathcal{D}$ consists of $T = 2{,}000$ transitions simulated from the closed-form optimal policy $k^*(k, z) = \alpha\beta zk^\alpha$. Since the optimal policy concentrates capital near its steady state, $\mathcal{D}$ covers only 11 of the 4,795 feasible (s, a) pairs—0.2% coverage—the distribution mismatch condition in **?**, Corollary 1.

Two algorithms are trained on $\mathcal{D}$.[19] BRM minimizes $\mathbb{E}_{\mathcal{D}}[(Q(s, a) - (r + \gamma \max_{a'} Q(s', a')))^2]$ where both $Q(s, a)$ and $Q(s', a')$ use the current network; gradients flow through both sides simultaneously. The key consequence is that the network can zero the residual at an observed pair (s, a) by co-moving $Q(s, a)$ and $Q(s', a')$ together, rather than moving either toward Q^*. This is the opposite of supervised learning, where labels are fixed external targets that do not move with the model weights. FQE prevents this by using a

[17] PopArt normalizes targets to have unit variance while adjusting the output layer so that the policy remains invariant to the normalization. PopArt allows consistent learning across reward scales spanning several orders of magnitude without reward clipping.

[18] **?** define a proxy as *unhackable* if increasing expected proxy return cannot decrease expected true return. Their main result states that for the set of all stochastic policies, two reward functions are unhackable only if one of them is constant. For deterministic policies and finite policy sets, non-trivial unhackable pairs exist, but the conditions are stringent.

[19] 50,000 gradient steps per seed, 3 seeds; two-layer MLP with 64 hidden units, Adam at 5×10^{-4}; target network updated every 500 steps.

frozen target network for $Q(s', a')$; the network must reduce $Q(s, a)$ toward a target that does not respond to its own gradient steps. Every 500 steps we record the Bellman error on $\mathcal{D}$ (current network on both sides) and the value error $\frac{1}{|\mathcal{F}|}\sum_{(s,a)\in\mathcal{F}} |Q_\theta(s, a) - Q^*(s, a)|$ over all 4,795 feasible pairs. As a supervised baseline, OLS regression of $\log c$ on $\log k$ and $\log z$ is estimated on expanding windows.[20]

The OLS baseline shows tight coupling between training and test loss (Pearson $r = -1.000$; Table **??**). The RL results reproduce **?** in the economic model: BRM achieves Bellman error 816× lower than FQE, yet both methods have nearly identical value error over the full MDP, yielding a VE/BE ratio three orders of magnitude higher for BRM. Both mechanisms from Section **??** operate: error cancellation on the 11 observed pairs and unconstrained $Q(s', a')$ at the 4,784 unobserved pairs.

Method	Final BE on $\mathcal{D}$	Final VE (all states)	VE/BE ratio
BRM (seed 42)	2.09×10^{-4}	7.061	33,808
BRM (seed 123)	2.91×10^{-4}	7.450	25,630
BRM (seed 777)	3.12×10^{-4}	7.278	23,320
BRM (mean)	2.71×10^{-4}	7.263	27,586
FQE (seed 42)	0.2037	6.927	34
FQE (seed 123)	0.2173	7.465	34
FQE (seed 777)	0.2425	7.281	30
FQE (mean)	0.2212	7.224	33
OLS ($n = 2{,}000$)	OOS MSE = 0.096	OOS $R^2 = 0.112$	$r = -1.000$

Table 1: Bellman error on dataset $\mathcal{D}$ and value error on the full MDP for BRM and FQE trained on offline Brock–Mirman data. BE is mean squared Bellman error evaluated with the current network on both sides; VE is mean absolute deviation from Q^* over all 4,795 feasible state-action pairs.

0.8 Discussion and Recommendations

Track episode return and policy entropy alongside training loss; entropy collapse and stagnating return are early warning signs of plasticity loss (Section **??**). Use PPO or SAC as default baselines before implementing custom algorithms. Report at least 10 seeds per configuration with IQM-based comparisons (Section **??**).

[20] Noise $\sigma = 0.30$; windows from $n = 10$ to 2,000; held-out test set of 500 transitions.

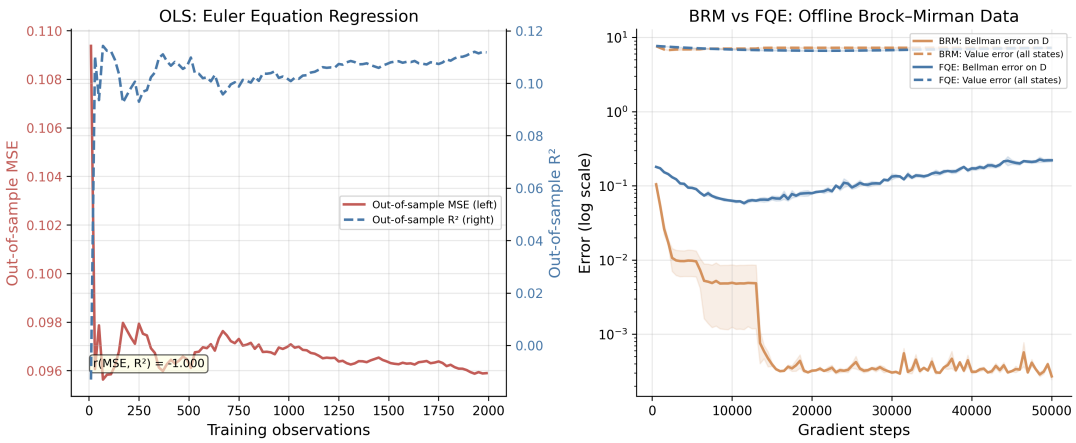

Figure 2: Left: OLS regression of log consumption on log capital and log productivity, estimated on expanding windows from the Brock–Mirman optimal policy. Out-of-sample MSE (left axis, red) and out-of-sample R^2 (right axis, blue) track each other with Pearson $r = -1.000$. Right: BRM (orange) and FQE (blue) trained on offline Brock–Mirman data $\mathcal{D}$ ($T = 2{,}000$ transitions, 0.2% state-space coverage); note the log scale on the y-axis. Solid lines show Bellman error on $\mathcal{D}$ (current network both sides); dashed lines show mean absolute value error against Q^* over all 4,795 feasible state-action pairs; shaded bands are ± 1 SE over 3 seeds.